

Angular Interview Questions and Answers

Part 1: Angular Basics

1. What is Angular?

Angular is a TypeScript-based open-source front-end framework developed by Google for building single-page web applications (SPAs). It uses a component-based architecture and provides powerful tools like data binding, dependency injection, and routing.

2. What are the main features of Angular?

Key features include: Component-based architecture, TypeScript support, Two-way data binding, Dependency Injection, Directives, Routing, and RxJS-based reactive programming.

3. Difference between AngularJS and Angular?

AngularJS (1.x) is JavaScript-based and uses MVC, while Angular (2+) uses TypeScript, is component-based, and has improved performance and modularity.

4. What is a Component in Angular?

A component is the basic building block of Angular apps. It consists of a TypeScript class (logic), HTML template (view), and CSS (styling). Example: `@Component` decorator defines a component.

5. What is a Module in Angular?

A module is a container for components, directives, pipes, and services. Every Angular app has a root module, typically `AppModule`. Modules help organize the application logically.

6. What is the purpose of `main.ts` and `AppModule`?

`main.ts` is the entry point of an Angular app—it bootstraps the root module (`AppModule`), which loads the main component (`AppComponent`).

7. What is Data Binding?

Data binding is the synchronization between the model and the view. Types: Interpolation `{{ }}`, Property binding `[ ]`, Event binding `( )`, and Two-way binding `[(ngModel)]`.

8. What are Directives?

Directives are classes that add behavior to elements. Types: Structural (`*ngIf`, `*ngFor`) and Attribute (`ngClass`, `ngStyle`).

9. What are Pipes in Angular?

Pipes transform data in templates. Example: `{{ user.name | uppercase }}`. You can create custom pipes using `@Pipe` decorator.

10. What is Dependency Injection?

Dependency Injection (DI) is a design pattern where Angular provides required services or objects to components automatically rather than creating them manually.

Part 2: Components & Lifecycle

11. Explain lifecycle hooks.

Lifecycle hooks allow you to tap into key events in a component's life. Examples: `ngOnInit()`, `ngOnChanges()`, `ngAfterViewInit()`, `ngOnDestroy()`.

12. Difference between `constructor()` and `ngOnInit()`

Constructor is used for dependency injection; `ngOnInit` is used for initialization logic after bindings are set.

13. Communication between components

Parent to Child: `@Input()`, Child to Parent: `@Output()` + `EventEmitter`, Sibling: Shared Service using `RxJS Subjects` or `BehaviorSubjects`.

Part 3: Routing

14. Role of Angular Router

Router enables navigation between views in a single-page application.

15. How to configure routes?

Define routes in `AppRoutingModule` using `RouterModule.forRoot()`.

16. What is Lazy Loading?

Lazy loading loads feature modules only when needed, improving performance.

17. How to pass parameters?

Use route parameters like `'/user/:id'` and access via `ActivatedRoute`.

18. What are Route Guards?

Guards like `CanActivate`, `CanDeactivate`, `CanLoad` protect routes and control access.

Part 4: Services & Dependency Injection

19. What is a Service?

A service provides reusable logic, usually for data access or business logic.

20. How to create and inject a service?

Use CLI 'ng g service serviceName' and inject via constructor.

21. What are Providers?

Providers define how to create or inject dependencies.

22. Difference between providedIn 'root' vs component?

'root' makes it singleton across app; at component level, it's instance-specific.

23. What is providedIn: 'root'?

It registers the service globally without adding it to providers array.

Part 5: HTTP & Observables

24. How to make HTTP calls?

Use HttpClient from HttpClientModule. Example: this.http.get('/api/users').

25. What is HttpClientModule?

It provides HttpClient service for making HTTP requests.

26. What are Observables?

Observables are streams used for async programming in Angular via RxJS.

27. Observable vs Promise?

Observables are lazy, cancellable, and can emit multiple values; Promises are eager and emit one.

28. Common RxJS operators?

map, filter, mergeMap, switchMap, debounceTime, catchError.

29. How to handle HTTP errors?

Use catchError operator and return a user-friendly error message.

Part 6: Forms

30. Template-driven vs Reactive forms?

Template-driven use directives, reactive forms are model-driven and more scalable.

31. How to create reactive form?

Use FormGroup, FormControl in TS and bind with [formGroup] in template.

32. How to validate forms?

Use built-in validators like Validators.required or custom validators.

33. What are FormGroup and FormControl?

FormGroup manages a group of controls; FormControl represents a single input.

34. Display validation messages?

Use *ngIf to show error messages conditionally.

Part 7: Advanced Topics

35. What is Change Detection?

Mechanism Angular uses to detect and update view when data changes.

36. What is Zone.js?

A library that tracks async operations to trigger change detection automatically.

37. What is ViewEncapsulation?

It controls CSS scoping. Types: Emulated (default), ShadowDom, None.

38. What are Standalone Components?

Introduced in Angular 14; don't need NgModule to declare.

39. Performance optimization techniques?

Use OnPush, Lazy Loading, Pure Pipes, trackBy in *ngFor.

40. OnPush vs Default?

OnPush detects changes only when inputs change; Default checks all bindings.

41. How to improve performance?

Lazy load routes, use async pipes, avoid deep change detection.

Part 8: Testing, Build & Deployment

42. What are Karma & Jasmine?

Karma is a test runner; Jasmine is a testing framework for writing specs.

43. How to unit test a component?

Use TestBed to configure testing module and write specs with Jasmine.

44. What is TestBed?

It configures and initializes components for testing in Angular.

45. What is angular.json?

Configuration file that defines project build, assets, and environments.

46. How to create production build?

Run 'ng build --prod' for optimized output.

47. What are environment files?

environment.ts for dev, environment.prod.ts for production settings.

48. What is AOT compilation?

Ahead-of-Time compiles templates at build time, reducing runtime overhead.